

1 faust2clap: Generating CLAP Plug-ins from Faust 2 DSP Code

3 **Facundo Franchino** ¹, **Stéphane Letz** ², and **Jatin Chowdhury** ³

4 ¹ School of Physics, Engineering and Technology, University of York, UK ² GRAME-CNCM, Lyon,
5 France ³ Department of Electrical Engineering and Computer Science, MIT, USA

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#) 
- [Repository](#) 
- [Archive](#) 

Editor: [Open Journals](#) 

Reviewers:

- [@openjournals](#)

Submitted: 01 January 1970

Published: unpublished

License

Authors of papers retain copyright
and release the work under a
Creative Commons Attribution 4.0
International License ([CC BY 4.0](#)).

6 Summary

7 faust2clap is a tool that implements a direct compilation route from Faust DSP source code
8 to plug-ins in the CLever Audio Plugin (CLAP) format. The system supports two modes of
9 operation: static compilation, which produces optimised standalone plug-ins, and dynamic
10 hot-reloading, which permits interactive development within the host environment. In the
11 dynamic mode, DSP code may be modified and recompiled while the host continues to run,
12 avoiding the repeated build-reload cycle that typically interrupts audio development workflows.

13 The implementation also provides automatic parameter discovery, MIDI support, polyphonic
14 voice allocation, and state persistence. These features make the tool suitable both for
15 researchers working with experimental DSP algorithms and for developers interested in rapid
16 prototyping of audio effects and synthesisers within a modern, open plug-in standard.

Statement of Need

18 The audio plug-in development ecosystem has traditionally required developers to navigate
19 complex APIs and manage significant boilerplate code when creating plug-ins for digital audio
20 workstations (DAWs). While Faust provides an elegant functional programming language for
21 DSP algorithm design ([Letz et al., 2018](#); [Orlarey et al., 2009](#)), and CLAP offers a modern,
22 open-source plug-in standard ([Free Audio Community, 2022](#)), there has been no direct path
23 between these two technologies.

24 faust2clap addresses this gap by providing researchers, audio developers, and educators with
25 a streamlined workflow for plugin development. The tool is particularly valuable for:

- 26 ▪ **Researchers** who wish to test novel DSP algorithms in DAWs without manually
27 implementing plugin infrastructure
- 28 ▪ **Educators** teaching DSP concepts who can now demonstrate algorithm behaviour directly
29 within professional audio software
- 30 ▪ **Audio developers** prototyping effects and synthesisers who benefit from Faust's compact
31 DSP language and immediate plug-in generation

32 The hot-reload capability is especially significant for iterative development, allowing real-time
33 experimentation with DSP parameters and algorithms while maintaining audio continuity in
34 the host application. This feature eliminates the traditional compile-reload-test cycle that
35 has historically slowed audio plugin development, enabling a more interactive and exploratory
36 approach to DSP design.

State of the Field

Several tools exist for generating audio plug-ins from Faust code, including `faust2vst`, `faust2au`, and `faust2lv2`, which target the VST, Audio Unit, and LV2 plug-in standards respectively (Letz et al., 2024). However, these tools generate only statically compiled plug-ins and lack support for the modern CLAP standard, which offers advantages including a stable C ABI, host-managed multithreading, per-voice control, fast and safe plug-in scanning, and an open-source license without patent encumbrances.

The CLAP standard itself, developed by Bitwig and u-he (Free Audio Community, 2022), introduces improvements over earlier standards in plug-in architecture design, but adoption has been limited by the lack of high-level development tools. While the `clap-helpers` library provides C++ utilities for CLAP development, it still requires substantial manual implementation work.

Related toolchains for generating plug-ins from high-level DSLs or patches include the Heavy Compiler Collection (`hvc`) (Wasted Audio, 2021), which converts Pure Data patches to C++ wrappers. Other established export-oriented environments include Cycling '74 RNBO for Max/MSP patches (Cycling '74, 2023). Adjacent deployment strategies, such as Camomile (Guillot, 2018), permit hosting patches within a plug-in environment rather than standalone compilation, which is more akin to the dynamic mode of `faust2clap`.

`faust2clap` extends the ecosystem through four contributions:

1. **First CLAP generator for Faust:** direct compilation of Faust DSP source code to CLAP plug-ins
2. **Dynamic hot-reload capability:** allows DSP code to be recompiled in real time without restarting the plug-in
3. **Automatic DSP type detection:** distinguishes between effects and instruments, configuring polyphonic behaviour accordingly
4. **GUI Support:** provides a Python interface for selecting DSP files and managing hot-reload operations

Implementation and Architecture

The implementation comprises two distinct compilation pathways, each addressing specific deployment requirements. The static mode produces standalone plug-ins through Faust's C++ code generation, whilst the dynamic mode provides runtime DSP compilation using `libfaust`'s interpreter backend.

The static implementation centres on a specialised CLAP architecture file (`clap-arch.cpp`) that adapts Faust's generated DSP classes to the CLAP API. A custom `CLAPMapUI` class extends Faust's `MapUI` to capture parameter metadata during instantiation, storing ranges, default values, and zone pointers as parameters are declared. The Python orchestration script (`faust2clap.py`) manages the compilation process by invoking Faust with architecture-specific flags and coordinating the CMake build chain. The script performs automatic DSP classification by analysing input and output port counts, distinguishing between effect processors and synthesiser instruments to configure appropriate polyphonic behaviour.

The dynamic implementation employs `libfaust`'s interpreter virtual machine to compile DSP code at runtime. File modification monitoring through the `efsw` library triggers recompilation cycles whilst preserving audio processing continuity. The system maintains parameter states across DSP updates and manages instance switching to avoid audio artifacts during code transitions. Compiled bytecode executes on a stack-based virtual machine with separate integer and floating-point heaps for DSP state management.

Both modes share infrastructure for audio buffer handling and MIDI event processing. Format

84 conversion between host audio representations (32-bit and 64-bit floating-point) and Faust's
85 FAUSTFLOAT type occurs transparently. Polyphonic operation utilises Faust's mydsp_poly
86 wrapper for voice allocation and MIDI event distribution. The build system integrates with
87 the Faust ecosystem through standardised paths obtained via `faust --archdir`, maintaining
88 compatibility across installation configurations.

89 Usage and Adoption

90 `faust2clap` has recently been merged into the main Faust repository, where it is now
91 distributed as part of the standard toolchain. Although still a new addition, its inclusion
92 makes the functionality available to the wider Faust user community. The tool has been
93 utilised in experimental settings to explore artificial reverberation algorithms in real-time DAW
94 environments, demonstrating its utility for iterative DSP research. Furthermore, developer
95 interest indicates strong potential for live coding workflows, which remain a subject for future
96 investigation. `faust2clap` provides a reference point for subsequent development of CLAP-
97 based tools and demonstrates the viability of high-level DSP specifications for modern plug-in
98 standards.

99 Acknowledgements

100 This work was supported by Google Summer of Code 2025. The author thanks Stéphane Letz
101 (GRAME-CNCM) for project supervision and continuous support, Jatin Chowdhury (MIT) for
102 CLAP expertise and mentorship, and the CLAP community for valuable feedback and testing.
103 Special recognition goes to the Faust development team at GRAME for maintaining the Faust
104 ecosystem that makes this work possible.

105 References

- 106 Cycling '74. (2023). *RNBO audio plugin export*. <https://rnbo.cycling74.com/>.
- 107 Free Audio Community. (2022). *CLAP: CLever Audio Plug-in API*. [https://github.com/free-](https://github.com/free-audio/clap)
108 [audio/clap](https://github.com/free-audio/clap).
- 109 Guillot, P. (2018). Camomile: Creating audio plugins with Pure Data. *Proceedings of the*
110 *Linux Audio Conference (LAC-18)*.
- 111 Letz, S., Orlarey, Y., & Fober, D. (2018). An overview of the faust developer ecosystem.
112 *Proceedings of the 1st International Faust Conference (IFC-18)*.
- 113 Letz, S., Orlarey, Y., & Fober, D. (2024). *Faust architecture files*. [https://faustdoc.grame.fr/](https://faustdoc.grame.fr/manual/architectures/)
114 [manual/architectures/](https://faustdoc.grame.fr/manual/architectures/).
- 115 Orlarey, Y., Letz, S., & Fober, D. (2009). Faust: An efficient functional approach to DSP
116 programming. In *New computational paradigms for computer music*. Delatour.
- 117 Wasted Audio. (2021). *Heavy compiler collection (hvcc)*. [https://wasted-audio.github.io/](https://wasted-audio.github.io/hvcc/)
118 [hvcc/](https://wasted-audio.github.io/hvcc/).